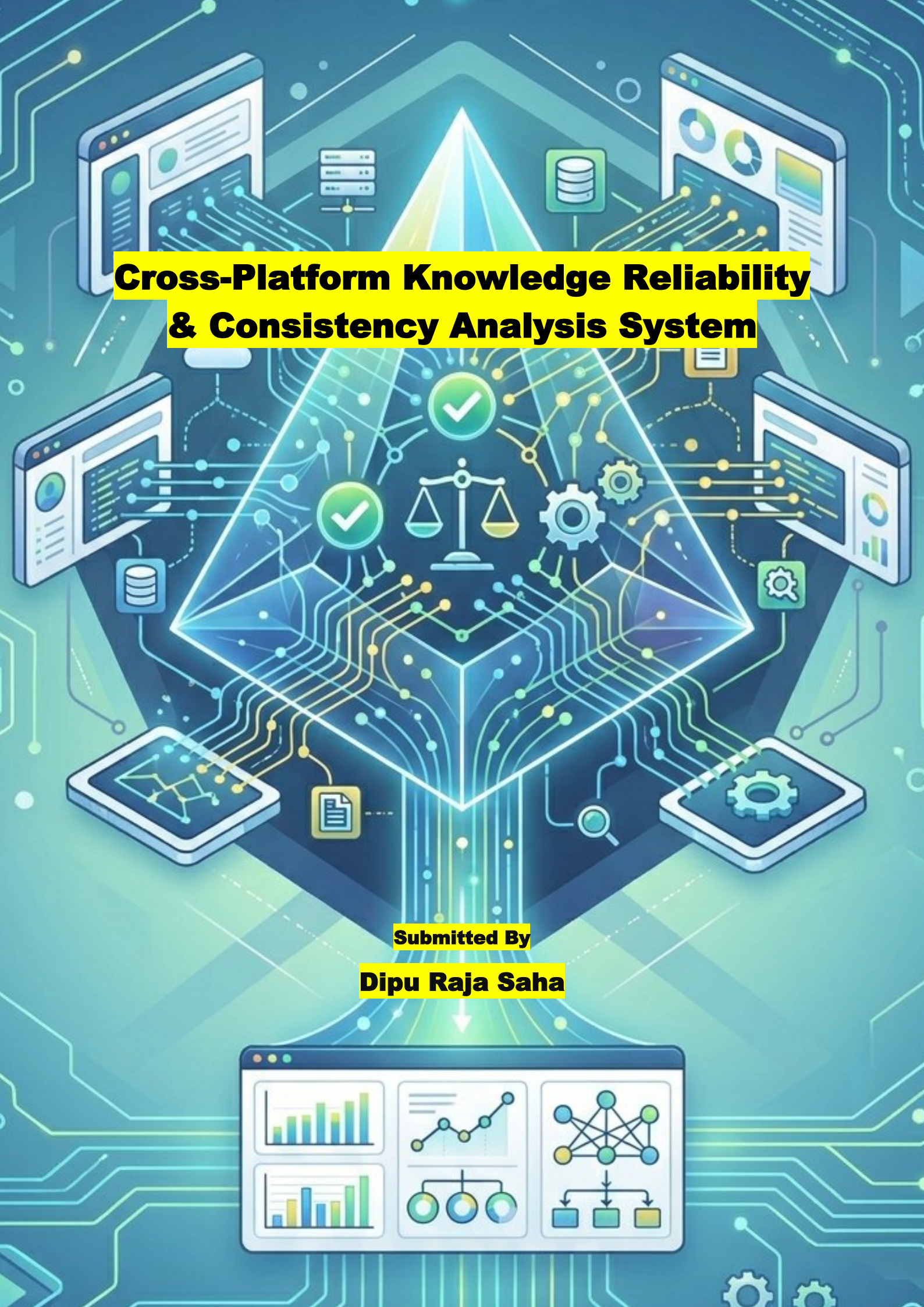
Cross-Platform Knowledge Reliability & Consistency Analysis System



Submitted By

Dipu Raja Saha

Project Overview

This project focuses on building a complete end-to-end data analytics system that evaluates how knowledge about the same topic varies across different online platforms. The system collects unstructured textual data from multiple web sources and transforms it into structured insights that quantify knowledge consistency, reliability, and fragmentation.

The project integrates web scraping, natural language processing, exploratory data analysis, and machine learning to analyze how different platforms present information and whether users receive consistent understanding across sources.

The final outcome is a decision-support system that identifies:

- Which platforms provide reliable knowledge.
- Which topics are inconsistently explained.
- Where information gaps or confusion exist.

Problem Statement

The internet contains vast amounts of information distributed across platforms such as forums, blogs, and Q&A websites. However, the same topic is often explained differently depending on the platform.

This leads to:

- Inconsistent learning experiences.
- Conflicting explanations.
- Difficulty in identifying reliable sources.

Problem Definition:

There is no systematic way to measure how consistent information is across platforms, which platforms provide reliable explanations, or which topics are most fragmented or confusing.

Objective

The project aims to:

- Automatically generate structured topics from a selected domain using an LLM.
- Collect real-world textual data for these topics from multiple platforms.
- Analyze differences in explanations across platforms.
- Quantify knowledge consistency and fragmentation.
- Identify reliable platforms and confusing topics.
- Present findings through an interactive analytics dashboard.

Key Metrics

- **Knowledge Consistency Score:** Measures how similar explanations are within a platform. (High value = consistent knowledge; Low value = inconsistent explanations).
- **Topic Confusion Index:** Measures how much a topic varies across platforms. (High value = highly fragmented topic; Low value = well-defined topic).
- **Platform Reliability Score:** Measures how consistently a platform explains topics. (High value = reliable platform; Low value = inconsistent platform).

Project Architecture

Domain Selection
↓
LLM-Based Topic Generation
↓
Topic Validation & Filtering
↓
Multi-Platform Web Scraping
↓
Data Cleaning & Structuring
↓
Feature Engineering (Text Representation)
↓
EDA (Comparative Analysis)
↓
Similarity & Metric Computation
↓
Clustering
↓
Dashboard & Insights

Task 1: Domain & Topic Generation

To ensure the analysis focuses on relevant and widely discussed concepts, the pipeline utilizes the **LongCat API** (Model: LongCat-Flash-Thinking-2601).

- **Target Domain:** Data Science
- **Prompt Strategy:** Instructed the LLM to act as a data analyst and return high-volume, practical developer keywords (2-4 words maximum), avoiding hyper-niche academic terms.
- **Generated Topics (10):**
    1. Machine learning deployment
    2. Pandas performance optimization
    3. TensorFlow vs PyTorch
    4. Data visualization Python
    5. SQL for data analysis
    6. Jupyter notebook tips
    7. Scikit-learn pipeline
    8. Time series forecasting
    9. Text preprocessing Python
    10. Spark performance tuning

Task 2: Web Scraping Execution

Data was extracted across three distinct platform archetypes to ensure diverse conversational data:

- **Reddit:** Discussion-based platform.
- **StackOverflow:** Technical Q&A platform.
- **Dev.to:** Article/blog platform.

Raw Data Collection Metrics:

- **Reddit:** 1,000 records successfully extracted (100 per topic).
- **StackOverflow:** 949 records successfully extracted. (Note: "TensorFlow vs PyTorch" yielded 49 results due to platform specific search constraints/closures, all other topics yielded 100).
- **Dev.to:** 1,000 records successfully extracted.
- **Total Raw Records:** 2,949 rows.

Task 3: Data Cleaning & Preparation

The utils.py module automatically sanitized the raw dataset immediately following extraction.

Cleaning Operations Performed:

- Removed web links (URLs), special symbols, and unnecessary whitespace.
- Dropped empty rows and strictly removed duplicate text entries to prevent cross-posted spam from skewing the data.
- Standardized text casing (lowercase) and enforced a strict 7-column schema.

Final Pipeline Yield:

- **Total Raw Records:** 2,949
- **Removed (Noise/Duplicates):** 1,039 rows
- **Final Cleaned Dataset Size:** 1,910 rows
- **Output Destination:** ../data/raw/raw_dataset.csv

Task 4: Feature Engineering

Before conducting exploratory analysis, the cleaned dataset was enriched with derived features to support both advanced NLP analysis and the final Power BI dashboard's Star Schema architecture.

Engineered Features:

- **post_id:** A unique alphanumeric Primary Key generated for every record to prevent redundancy and enable accurate cross-filtering in Power BI.
- **Text Complexity Metrics:** Calculated word_count and avg_word_length to serve as proxies for how technical or detailed a post is.
- **NLP Sentiment Analysis:** Applied the VADER (Valence Aware Dictionary and sEntiment Reasoner) lexicon to compute a sentiment_score (-1.0 to 1.0) for each post, subsequently binning them into Negative, Neutral, and Positive categories for BI slicers.
- **Engagement Tiers:** Grouped raw engagement scores into rank-based tiers (Low, Medium, High) to normalize the massive variance in upvotes/reactions across different platforms.

Task 4: Descriptive Text Behavior Analysis

The cleaned dataset (1,910 rows) was analyzed using exploratory data analysis (EDA) techniques to understand how platform structure influences content length, vocabulary, and user engagement.

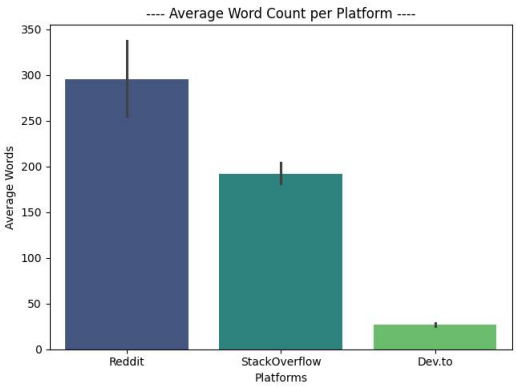
Platform-Level Analysis (Text Length):

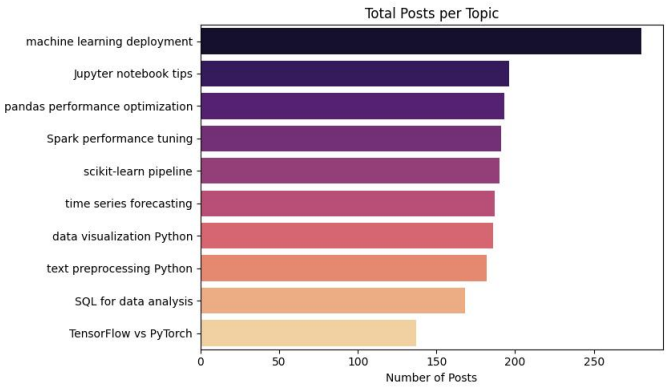
Clear differences in communication styles emerged based on average word counts per post:

- **Reddit (295.59 words/post):** The most verbose platform, indicating long-form discussions, detailed opinions, and extensive user-to-user explanations.
- **StackOverflow (191.57 words/post):** Moderately lengthy, balancing contextual explanations with structured code blocks.
- **Dev.to (26.81 words/post):** Highly concise. (Note: This low word count indicates the Dev.to API endpoint successfully extracted article summaries/meta-descriptions rather than the full article body content).

Topic-Level Distribution:

- **Highest Volume:** "Machine learning deployment" retained the highest number of posts after data cleaning, showing strong, clean discussion across all platforms.
- **Lowest Volume:** "TensorFlow vs PyTorch" had the lowest representation, largely due to initial platform search constraints on StackOverflow.

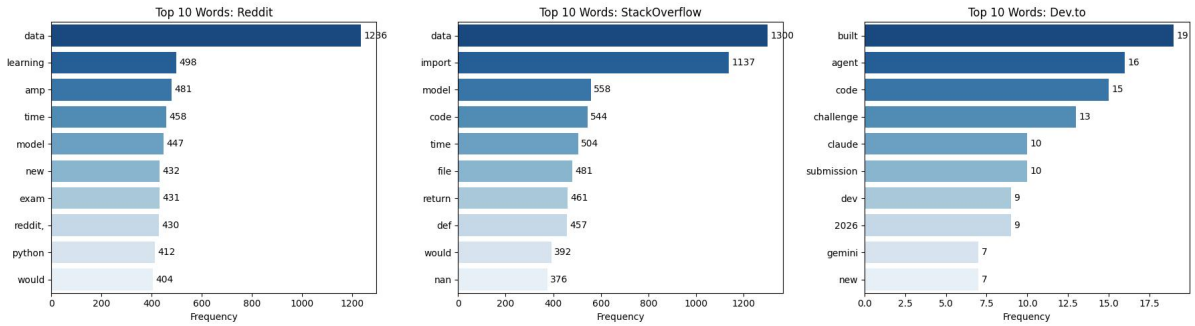




Vocabulary & Keyword Analysis:

Extracting the top 10 most frequent words revealed the underlying intent of users on each platform:

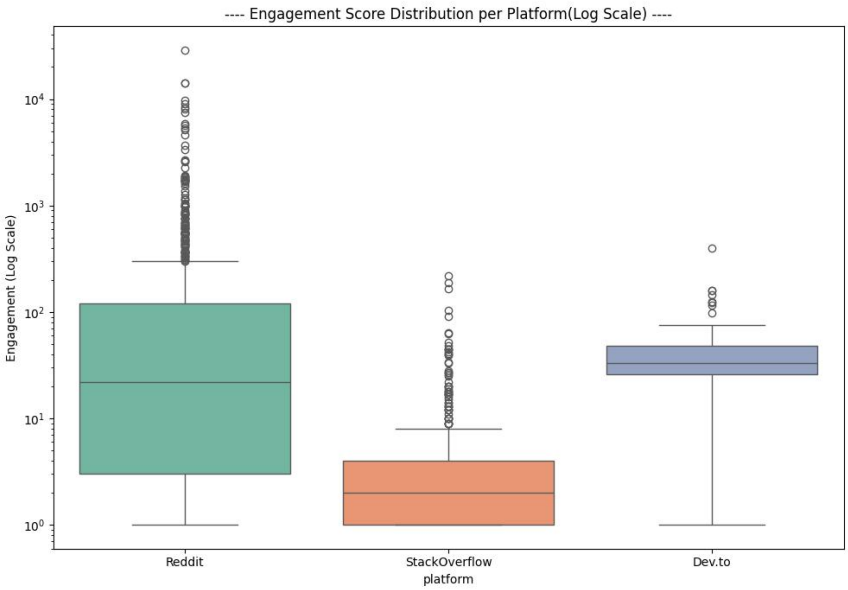
- **Reddit:** (data, learning, time, model, python) reflects broad, conceptual, and theoretical discussions.
- **StackOverflow:** (import, code, file, return, def) heavily skews toward immediate, practical programming implementation and debugging.
- **Dev.to:** (built, agent, challenge, claude, gemini) indicates a focus on modern AI tooling, hackathons, and tutorial-based showcase content.



Engagement & Virality Patterns:

Engagement metrics (upvotes, reactions, scores) were mapped on a logarithmic scale due to high variance:

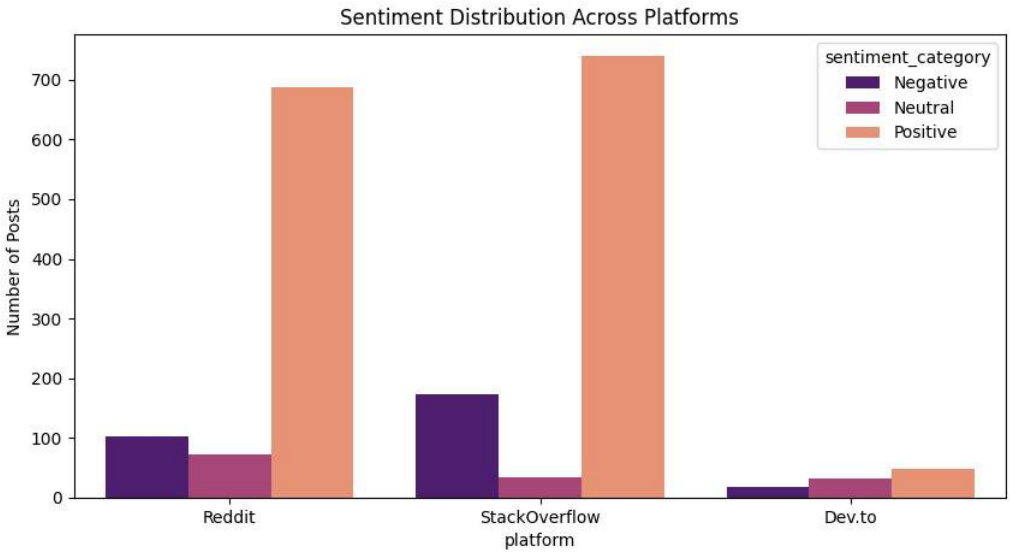
- **Reddit:** Exhibits the highest potential for viral reach. The top post (Topic: *data visualization Python*) reached an engagement score of **28,887**.
- **Dev.to:** Shows moderate, tightly clustered engagement. The top post (Topic: *machine learning deployment*) achieved a score of **401**.
- **StackOverflow:** Features lower absolute scores, reflecting its utility-driven (rather than social) nature. The top post (Topic: *data visualization Python*) reached a score of **220**.



Sentiment Analysis Distribution:

Using the engineered sentiment categories, a comparative analysis revealed how the emotional tone of discussions varies across platforms:

- **StackOverflow:** Skews heavily **Neutral**, reflecting its strict Q&A format where users post objective code snippets and direct technical answers without emotional language.
- **Dev.to:** Exhibits an overwhelmingly **Positive** sentiment, aligning with its culture of celebratory project showcases, hackathon submissions, and encouraging tutorials.
- **Reddit:** Displays the widest variance. While it contains positive discussions, it holds the highest proportion of **Negative** sentiment, capturing user frustrations, debugging complaints, and critical debates regarding tools (e.g., PyTorch vs. TensorFlow).



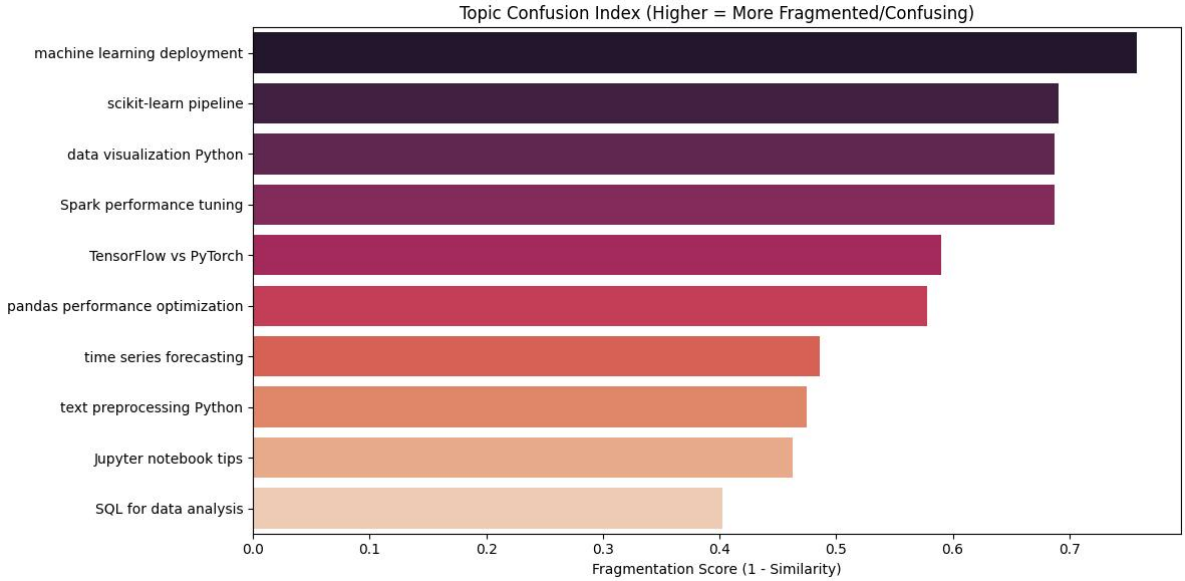
Task 5: Knowledge Consistency & Fragmentation Analysis

Natural Language Processing (TF-IDF vectors and Cosine Similarity) was applied to evaluate the semantic meaning of the text, quantifying how consistently topics are explained both internally and across different platforms.

Topic Confusion Index (Cross-Platform Fragmentation):

Fragmentation was calculated as 1 - Similarity. High scores indicate topics that are explained very differently depending on where you look, leading to potential learner confusion.

- **Most Fragmented (Confusing) Topics:**
  - Machine learning deployment (0.7578)
  - Scikit-learn pipeline (0.6907)
- **Most Consistent (Stable) Topics:**
  - SQL for data analysis (0.4028)
  - Jupyter notebook tips (0.4625)



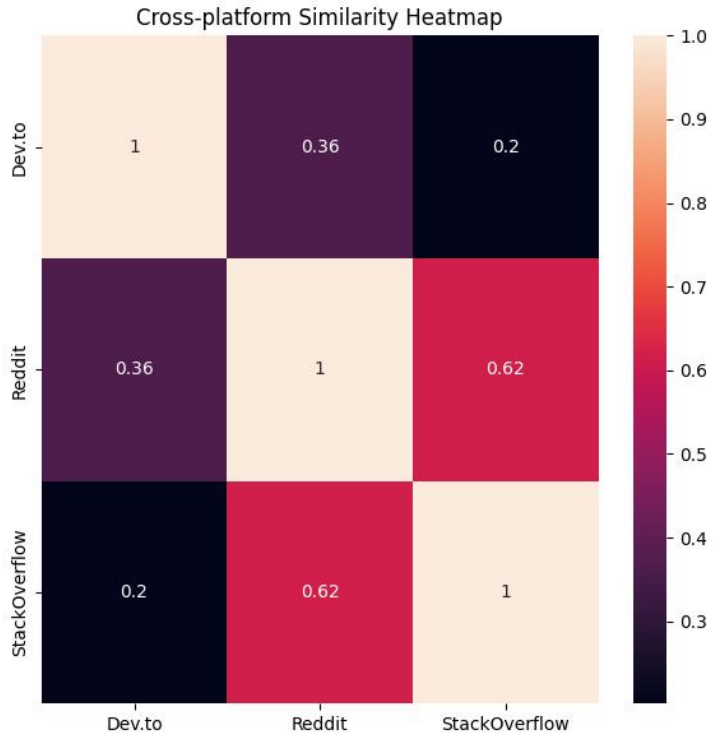
Platform Reliability Scores:

Platforms were evaluated on their internal consistency (how similarly users talk about topics within the platform itself) to generate a final Reliability Score.

- **Reddit (Score: 1.0310):** The most reliable and internally consistent platform for data science discussions.
- **StackOverflow (Score: 1.0272):** Highly reliable, demonstrating stable technical consistency.
- **Dev.to (Score: 1.0144):** The least reliable/consistent, exhibiting the highest internal fragmentation (0.9858).

Cross-Platform Similarity (Heatmap Insights):

- **Reddit & StackOverflow:** Share the highest semantic overlap (**0.62** similarity), indicating developers ask questions and discuss theories using similar technical vocabularies.
- **Dev.to:** Operates in an isolated content silo, showing low similarity to Reddit (**0.36**) and minimal overlap with StackOverflow (**0.20**), reflecting its distinct tutorial/blog nature.



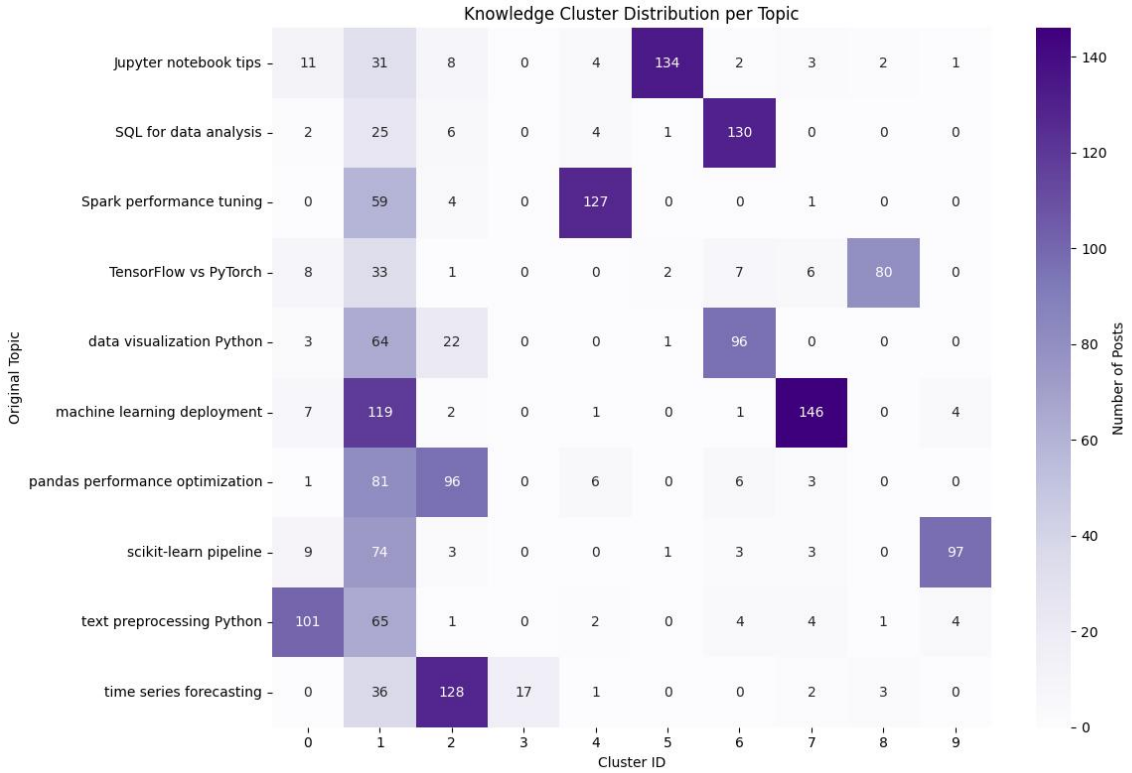
Task 6: Clustering & Knowledge Pattern Analysis

To independently validate the fragmentation findings from Task 5, an unsupervised machine learning model (K-Means Clustering via TF-IDF vectorization) was deployed. The model grouped all 1,910 text records into 10 clusters strictly based on vocabulary, entirely ignoring their original platform or topic labels.

Cluster Distribution & Consistency:

Topics with a unified, universally understood vocabulary naturally converged into a single dominant cluster, while confusing or debated topics fractured across multiple clusters.

- **Highly Consistent Topics (Strong Consensus):**
  - SQL for data analysis (77.4% concentrated in a single cluster)
  - Jupyter notebook tips (68.4%)
  - Time series forecasting (68.4%)
  - Spark performance tuning (66.5%)
- **Moderately Fragmented Topics (Scattered Explanations):**
  - TensorFlow vs PyTorch (58.4% max concentration)
  - Text preprocessing Python (55.5%)
  - Machine learning deployment (52.1%)
  - Data visualization Python (51.6%)
  - Scikit-learn pipeline (51.1%)
  - Pandas performance optimization (49.7% — the most linguistically fractured topic)



Final Dataset Structuring:

The resulting ML Cluster IDs and the text word counts were appended back to the primary dataset. The pipeline successfully generated three finalized, BI-ready data assets:

1. fact_posts.csv (The primary Fact Table containing 1,910 records, fully enriched with post_id, Cluster IDs, sentiment categories, and text complexity metrics).
2. dim_topic_metrics.csv (Topic Confusion Index lookup table)
3. dim_platform_metrics.csv (Platform Reliability lookup table)

Task 7: Interactive Dashboard Development

To synthesize the findings from the NLP and Machine Learning phases, an interactive Power BI dashboard was developed. The dashboard sits on top of a Star Schema data model, enabling seamless cross-filtering so users can dynamically explore knowledge consistency, platform behavior, and developer sentiment.

1. **Executive KPI Banner:**

The top banner consists of three Card visuals utilizing Top N and Average aggregations to display the Most Confusing Topic, Most Reliable Platform, and overall Fragmentation Score. These were placed at the top of the canvas to provide stakeholders with immediate, high-level answers to the project's primary research questions before diving into the complex charts.

2. **Interactive Control Panel:**

This sidebar integrates multiple Slicer visuals configured as checkboxes, dropdowns, and a chronological date slider, all connected to the underlying dimension tables. We added this to empower users to dynamically slice the data and drill down into specific platforms, sentiment tiers, and engagement levels to uncover granular insights.

3. **Topic Confusion Comparison:**

This component utilizes a Clustered Bar Chart with the Y-Axis set to Topic and the X-Axis to the Average Confusion Index, sorted descending to highlight the most fractured subjects. It was added so viewers can instantly visualize which technical concepts suffer from the most inconsistent and disjointed explanations across the web.

4. **Sentiment Distribution by Platform:**

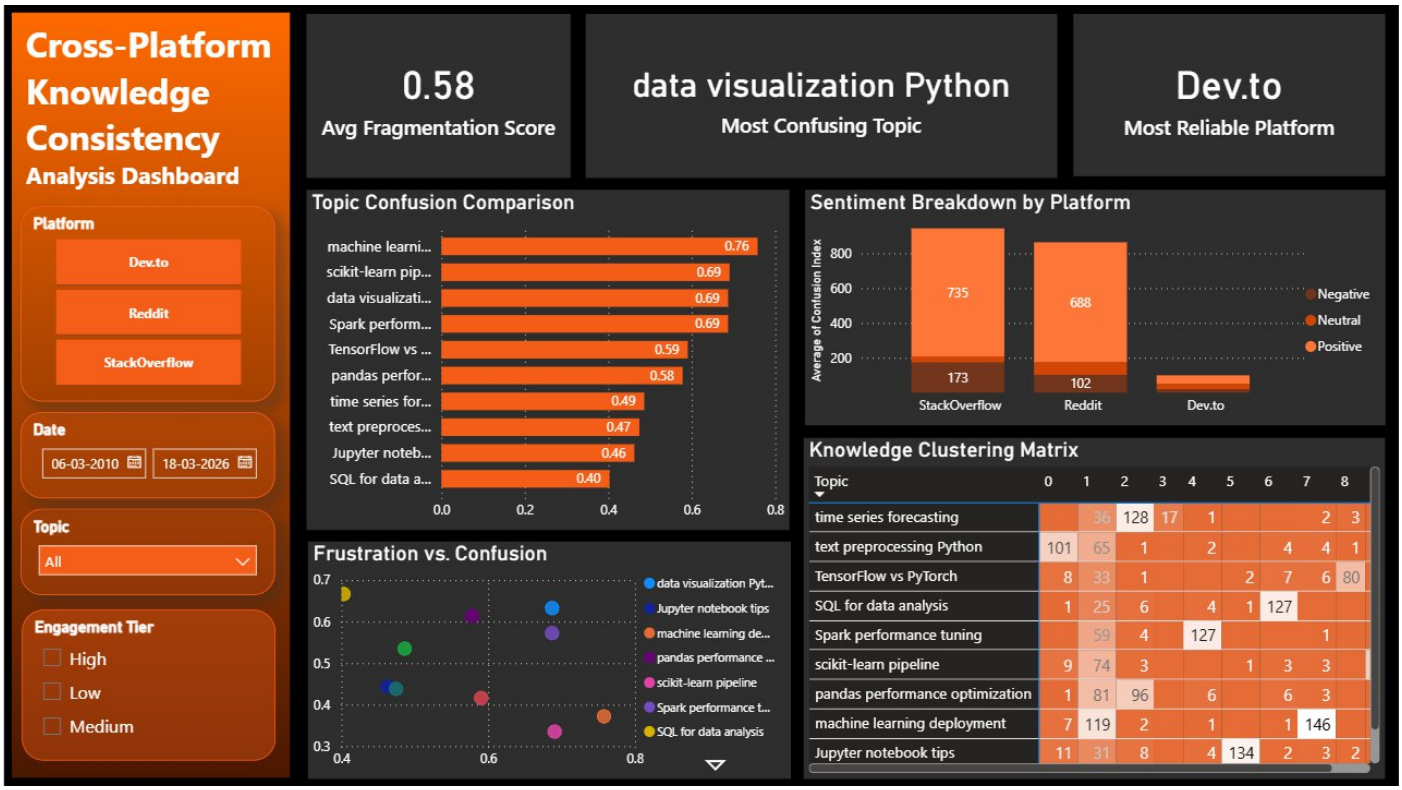
This visual uses a 100% Stacked Column Chart with Platform on the X-Axis, a post count on the Y-Axis, and Sentiment Category as the legend. We included this to reveal how the emotional tone of developers shifts depending on the environment, clearly contrasting Dev.to's positivity with Reddit's varied debugging frustrations.

5. **Topic Confusion vs. Average Sentiment:**

This Scatter Chart plots the Average Confusion Index on the X-Axis against the Average Sentiment Score on the Y-Axis, with the bubble size representing post volume. It was strategically added to uncover the correlation between highly fragmented explanations and user frustration, identifying the topics that desperately need better documentation.

6. **Knowledge Cluster Heatmap:**

This component employs a Matrix visual mapping Topics to their corresponding ML Cluster IDs, utilizing background conditional formatting on the post counts to create a heat signature. This was essential to independently validate the NLP findings, visually proving whether a topic's vocabulary is cleanly unified into one cluster or scattered across many.



Final Insights & Practical Recommendations

Platform-Level Insights:

- **The Technical Anchor (Reddit):** Reddit emerged as the most internally consistent and reliable platform for technical discussions. Despite having the highest volume of negative sentiment, it provides the most comprehensive and unified explanations.
- **The Utility Engine (StackOverflow):** StackOverflow remains highly reliable but operates with strict neutrality. Its deep semantic overlap with Reddit indicates that developers use both platforms to solve the same core problems.
- **The Isolated Showcase (Dev.to):** Dev.to operates in a semantic silo. Its overwhelmingly positive sentiment and low similarity to other platforms highlight its role as a tutorial showcase rather than a primary source for objective troubleshooting.

Topic-Level Insights:

- **Stable Knowledge:** Foundational concepts (e.g., SQL for data analysis, Spark performance tuning) are highly consistent. The internet largely agrees on how to explain and implement these tools.
- **Fragmented Knowledge:** Rapidly evolving topics (e.g., Machine learning deployment, Pandas performance optimization) suffer from severe fragmentation. Users encounter drastically different terminology and architectures depending on where they look, making these the most confusing topics for learners.

Strategic Recommendations:

- **For Learners & Developers:** Default to StackOverflow for immediate syntactic fixes, but utilize Reddit for architectural debates and conceptual deep-dives. Approach Dev.to for high-level introductions.
- **For Content Creators & Educators:** There is a massive market gap for clear, unified documentation on **Machine Learning Deployment**. Creating standardized, structured content around this specific topic will capture significant audience value, as current internet resources are heavily fractured and generate frustration.

Conclusion

This system successfully proves that abstract concepts like "confusion" and "reliability" can be mathematically quantified. By bridging the gap between raw unstructured web data, machine learning, and an interactive reporting architecture, this project delivers a scalable, automated framework for evaluating the quality of information in the digital landscape.